



UNIVERSIDAD DE QUINTANA ROO

ACP-137 INTRODUCCIÓN A LA MECATRÓNICA

MANUAL DE PRÁCTICAS DE LABORATORIO

M.M. Jesús Orifiel Alvarez Ruiz

Dr. Freddy Ignacio Chan Puc

Documento aprobado en reunión del área de ingeniería en Sistemas de Energía

noviembre de 2021

CONTENIDO

Índice de figuras.....	4
Introducción.....	1
Práctica I- Control del encendido de LEDs.....	2
Objetivo de la unidad:.....	2
Objetivos de la práctica:.....	2
Antecedentes	3
Software necesario.....	7
Desarrollo de la práctica.....	7
Resultados.....	10
Discusión.....	14
Conclusión.....	14
PRÁCTICA II- Voltímetro y Amperímetro de Corriente Directa.....	15
Objetivo de la unidad	15
Objetivos de la práctica.....	15
Antecedentes.....	16
Software Necesario	18
Desarrollo de la práctica	19
Resultados.....	21
Discusión	26
Conclusión	26
PRÁCTICA III- Ejercicios de Señales Electricas	27
Objetivo de la unidad:.....	27
Objetivos de la práctica:.....	27
Antecedentes.....	28

CONTENIDO

Software necesario.....	28
Desarrollo de la práctica	28
Resultados	31
Discusión	32
Conclusiones	32
PRÁCTICA IV- Fuentes de Corriente Directa	33
Objetivo de la unidad	33
Objetivos de la práctica.....	33
Antecedentes.....	33
Software Necesario	37
Desarrollo de la práctica	37
Resultados	40
Discusión	41
Conclusión	41
PRÁCTICA V- Fuentes de Corriente Directa	42
Objetivo de la unidad	42
Objetivos de la práctica.....	42
Antecedentes.....	42
Software Necesario	42
Desarrollo de la práctica	43
Resultados	45
Discusión	45
Conclusión	46
Bibliografía.....	46

Índice de figuras.

- Figura 1. Resistores en conexión serie **¡Error! Marcador no definido.**
- Figura 2. Resistores en conexión paralelo **¡Error! Marcador no definido.**
- Figura 3. Como seleccionar modo de diseño de circuitos **¡Error! Marcador no definido.**
- Figura 4. Como crear un nuevo circuito **¡Error! Marcador no definido.**
- Figura 5. Como seleccionar un protoboard **¡Error! Marcador no definido.**
- Figura 6. Como seleccionar un resistor **¡Error! Marcador no definido.**
- Figura 7. Selección de valor del componente resistivo **¡Error! Marcador no definido.**
- Figura 8. Selección de modo de uso del multímetro **¡Error! Marcador no definido.**
- Figura 9. Ubicación del botón para iniciar simulación **¡Error! Marcador no definido.**
- Figura 10. Ubicación de opción para todos los componentes **¡Error! Marcador no definido.**
- Figura 11. Selección de una fuente de energía de corriente directa **¡Error! Marcador no definido.**
- Figura 12. Selección de los parámetros de la salida de la fuente **¡Error! Marcador no definido.**
- Figura 13. Selección de generador de funciones **¡Error! Marcador no definido.**
- Figura 14. Selección de osciloscopio **¡Error! Marcador no definido.**
- Figura 15. Inserción de parámetros de la señal **¡Error! Marcador no definido.**
- Figura 16. Inserción de parámetros del osciloscopio **¡Error! Marcador no definido.**
- Figura 17. Selección de módulos fotovoltaicos **¡Error! Marcador no definido.**

CONTENIDO

Figura 18. Selección de características de los módulos fotovoltaicos.... **¡Error! Marcador no definido.**

Figura 19. Selección de medidor **¡Error! Marcador no definido.**

Figura 20. Selección de modo de operación del medidor **¡Error! Marcador no definido.**

Figura 21. Selección de modo de operación del medidor **¡Error! Marcador no definido.**

Introducción

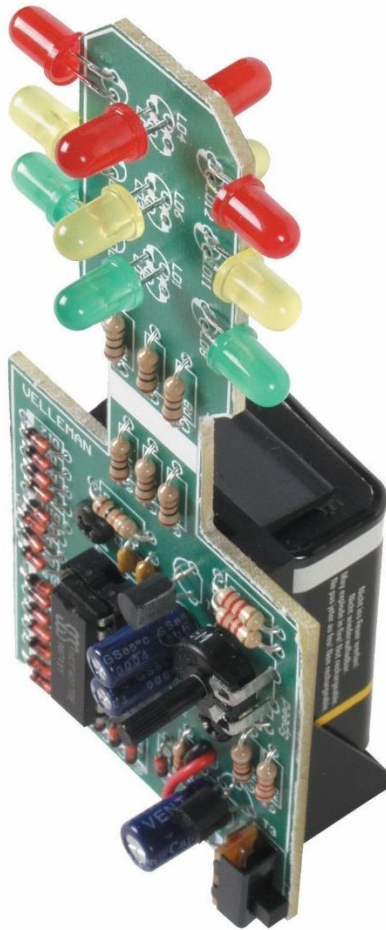
En este documento se presenta el “Manual de Prácticas de Laboratorio” de la asignatura “ACP-137 Introducción a la Mecatrónica”, la cual cubre tópicos relacionados como: microcontroladores, programación de microcontroladores, integración de sensores analógicos y digitales con microcontroladores, e integración de actuadores controlados por microcontroladores.

Este manual de prácticas integra cinco prácticas de laboratorio, con la finalidad de reforzar el aprendizaje teórico del estudiante, llevándolo a realizar diferentes prácticas sobre la construcción de sistemas electrónicos con base en los microcontroladores Arduino.

De esta forma, la práctica 1 consiste en la construcción y programación de un semáforo. La práctica 2 consiste en el control LEDs RGB (Red-Green-Blue), y la comunicación serial con un microcontrolador. La práctica 3 consiste en lectura de un puerto. La práctica 4 consiste en la medición de distancia con un sensor ultrasónico. La practica 5 consiste en la detección de movimiento con el sensor PIR (Passive Infrared)

Con la finalidad que este material sea de utilidad en la formación profesional de los alumnos del programa educativo de Ingeniería en Sistemas de Energía.

Práctica I- Control del encendido de LEDs



Objetivo de la unidad:

Describir el funcionamiento de los comandos de programación, los microcontroladores, y los componentes asociados, así como el manejo de los mismos

Objetivos de la práctica:

- El alumno practicará los comandos básicos para el control del encendido de LEDs en una secuencia definido utilizando un microcontrolador

¿Qué es Arduino?

Arduino es una plataforma electrónica de código abierto basada en hardware y software fáciles de usar. Las placas Arduino pueden leer entradas (luz en un sensor, un dedo en un botón o un mensaje de Twitter) y convertirlo en una salida, activando un motor, encendiendo un LED, publicando algo en línea. Puede decirle a su tablero qué hacer enviando un conjunto de instrucciones al microcontrolador en el tablero. Para hacerlo, utiliza el lenguaje de programación Arduino (basado en Wiring) y el Software Arduino (IDE), basado en Processing.

Arduino uno



Arduino Uno es una placa de microcontrolador basada en ATmega328P ([hoja de datos](#)). Tiene 14 pines de entrada / salida digital (de los cuales 6 se pueden usar como salidas PWM), 6 entradas analógicas, un resonador cerámico de 16 MHz (CSTCE16M0V53-R0), una conexión USB, un conector de alimentación, un encabezado ICSP y un botón de reinicio. Contiene todo lo necesario para soportar el microcontrolador; simplemente conéctelo a una computadora con un cable USB o enciéndalo con un adaptador de CA a CC o una batería para comenzar.

"Uno" significa uno en italiano y fue elegido para marcar el lanzamiento de Arduino Software (IDE) 1.0. La placa Uno y la versión 1.0 del software Arduino (IDE) fueron las versiones de referencia de Arduino, ahora evolucionadas a versiones más recientes. La placa Uno es la primera de una serie de placas USB Arduino y el modelo de referencia para la plataforma Arduino; Para obtener una lista extensa de placas actuales, pasadas o desactualizadas, consulte el índice de placas Arduino.

Comandos básicos

`#define` es un componente útil de C ++ que permite al programador dar un nombre a un valor constante antes de compilar el programa. Las constantes definidas en Arduino no ocupan espacio de memoria de programa en el chip. El compilador reemplazará las referencias a estas constantes con el valor definido en el momento de la compilación.

Sintaxis

```
#define constantName value
```

```
Ejemplo: #define ledPin 3
```

`digitalWrite();`

Escribe un valor **HIGH** o un valor **LOW** en un pin digital. Si el pin ha sido configurado como **OUTPUT** con **pinMode()**, su voltaje se establece en el valor correspondiente: 5V (o 3.3 V en placas de 3.3 V) para HIGH, 0 V (masa) para LOW.

Sintaxis

```
digitalWrite(pin, value)
```

Parámetros

pin: el número de pin

value: HIGH o LOW

Retorno

Ninguno

Ejemplo

```
digitalWrite(ledPin, HIGH);    // enciende el LED
```

`pinMode();`

Configura el pin especificado para comportarse como una entrada o como una salida. Ver la descripción de digital pins para ver detalles sobre la funcionalidad de los pines.

A partir de Arduino 1.0.1, es posible activar las resistencias pull-up internas con el modo INPUT_PULLUP. Además, el modo INPUT desactiva de forma explícita las pull-ups internas.

Sintaxis

```
pinMode(pin, mode)
```

Parámetros

pin: el número de pin cuyo modo queremos configurar

mode: INPUT, OUTPUT, o INPUT_PULLUP. (ver la página digital pins para una descripción más completa de esta funcionalidad.)

Retorno

Ninguno

Ejemplo

```
pinMode(ledPin, OUTPUT); // configura el pin como salida
```

```
digitalRead();
```

Lee el valor de un pin digital especificado, puede ser HIGH o LOW.

Sintaxis

```
digitalRead(pin)
```

Parámetros

pin: el número de pin digital que se quiere leer (int)

Retornos

HIGH o LOW

Ejemplo

```
val = digitalRead(inPin); // lee el pin de entrada
```

Condicional `if`

La sintaxis de la sentencia `if` con Arduino es muy sencilla. Comenzamos escribiendo la palabra reservada `if` (en español se traduce como si condicional). Luego entre paréntesis ponemos la condición y por último abrimos y cerramos las llaves.

Sintaxis

```
if(condición) {  
    //todo lo que pongamos qui se ejecutar  
    //solo si se cumple la condición  
}
```

Ejemplo

```
if(i>2){  
digitalWrite (ledPIN, HIGH);  
}
```

Bucle `for`

Se utiliza para repetir un bloque de declaraciones entre llaves. Por lo general, se usa un contador de incrementos para incrementar y terminar el ciclo. Es útil para cualquier operación repetitiva y, a menudo, se usa en combinación con matrices para operar en colecciones de datos / pines.

Sintaxis

```
for (initialization; condition; increment) {  
    // statement(s);  
}
```

Ejemplo de código

```
// Dim an LED using a PWM pin  
int PWMpin = 10; // LED in series with 470 ohm resistor on pin 10  
  
void setup() {  
    // no setup needed  
}
```

```
void loop() {  
  for (int i = 0; i <= 255; i++) {  
    analogWrite(PWMPin, i);  
    delay(10);  
  }  
}
```

Software necesario

- TinkerCAD en línea (<https://www.tinkercad.com/>)
- Computadora con Windows o iOS

Materiales

- 1 Arduino Uno con conexión a USB
- 2 Led's de color rojo
- 2 Led's de color amarillo
- 2 Led's de color verde
- 6 resistencias de 330 Ohm
- 1 botón pulsador
- 1 resistencia de 10 kOhm
- Cables de conexión.

Desarrollo de la práctica

1. La práctica 1.1 fue leída y comprendida
2. Un buscador fue utilizado para llegar a la página web de TinkerCAD
3. El botón de circuitos fue pulsado. La ubicación del botón es demostrada en la figura 1

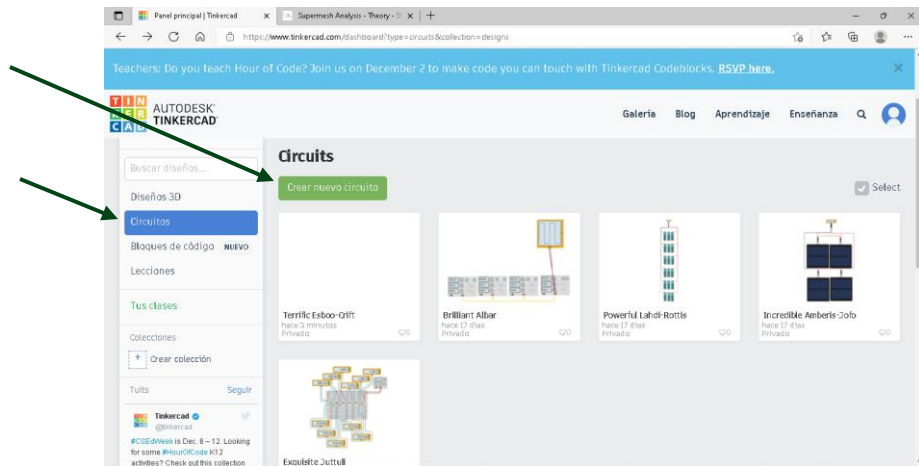


Figura 1. Seleccionar la opción de nuevo circuito

4. La opción de crear un nuevo circuito fue tomada pulsando el botón nombrado “Crear nuevo circuito” cuya ubicación es demostrado en la figura 1
5. La barra de componentes fue cambiada de componentes “básicos” a “todos”. La celda donde este cambio es hecho esta indicada en la ilustración 2

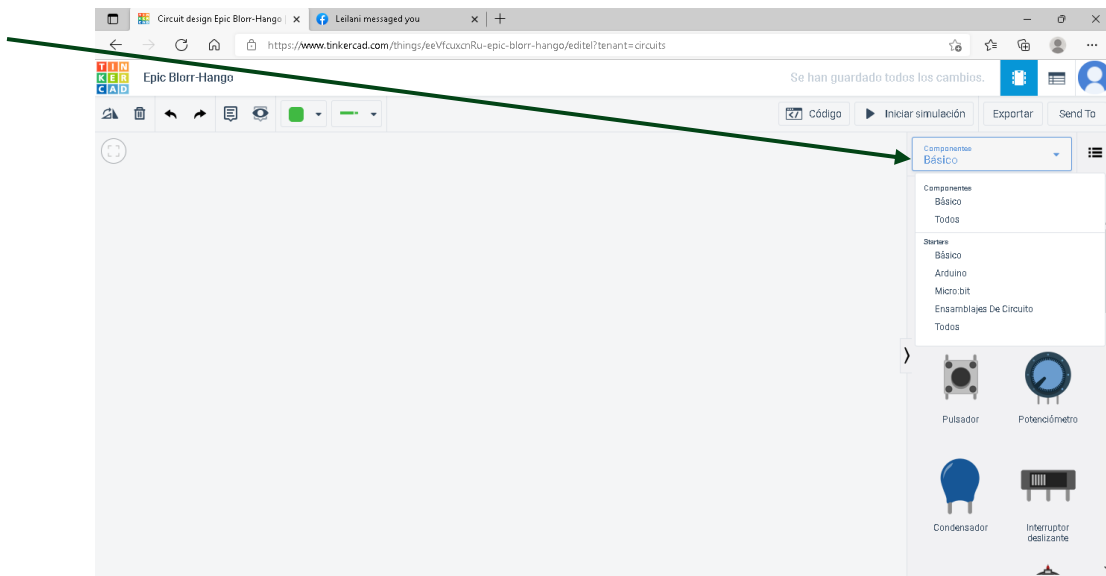


Figura 2. Selección disponibilidad de componentes

6. Los componentes requeridos para la practica uno fueron seleccionados y puestos en el área de trabajo
7. El circuito exigido por el ejercicio 1.1 fue construido como es indicado en la figura 3

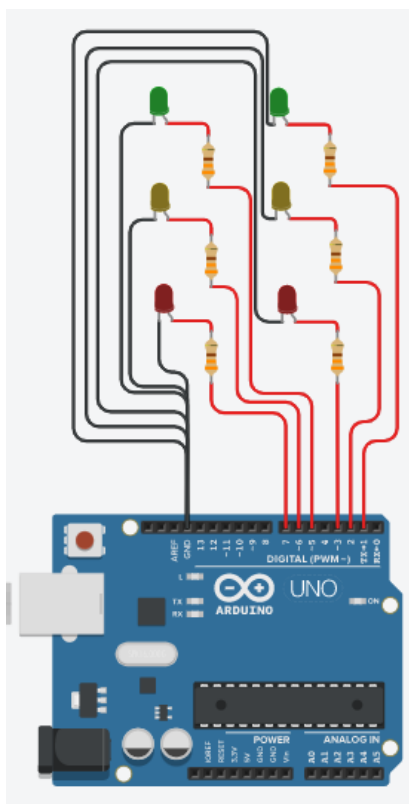


Figura 3. El circuito de dos semáforos

8. La ventana para escribir código fue abierta picando el botón nombrado “Código”. La ubicación de este en la pantalla es tal como lo demuestra figura 4

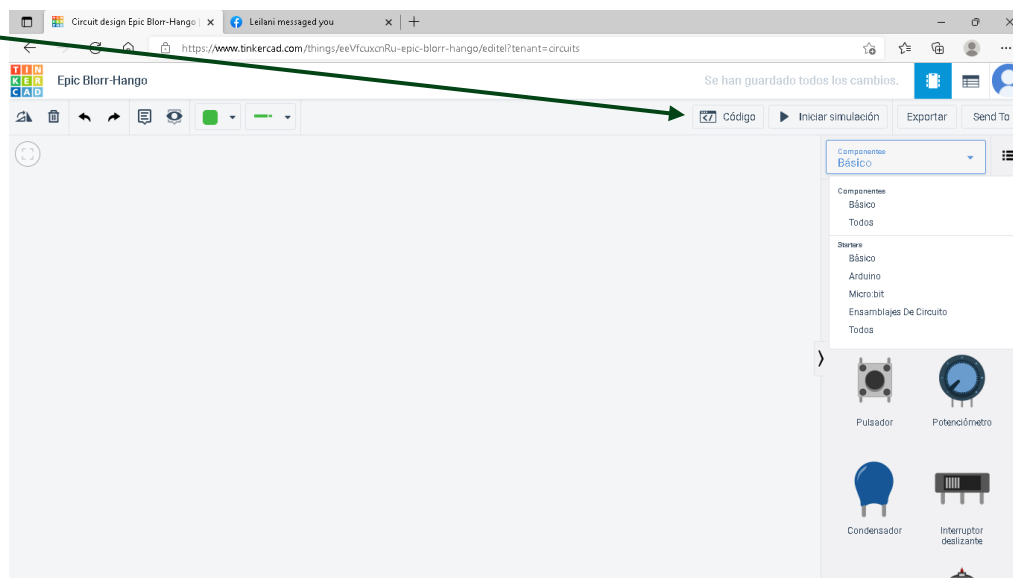


Figura 4. Pestaña para programar el Arduino

9. El código fue preparado según las instrucciones indicado en el ejercicio 1.1. La ventana de programación es demostrada en la figura 5

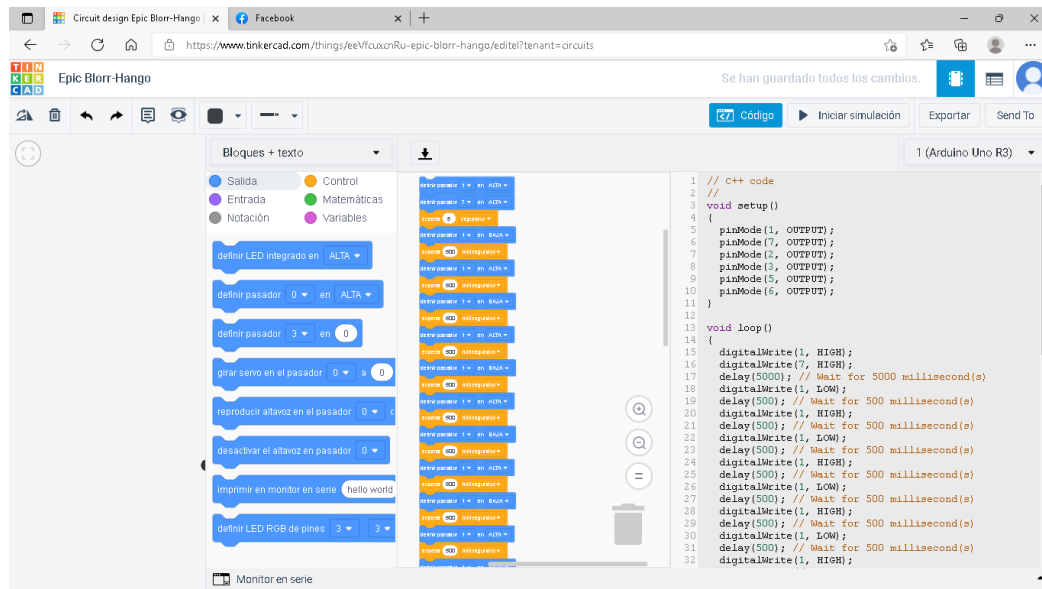


Figura 5. Ventana de programación

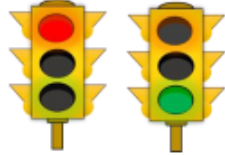
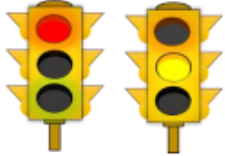
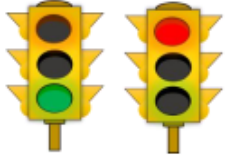
10. Los resultados de la practica fueron documentados mediante capturas de pantalla en la sección de resultados
11. Pasos 1 a 10 fueron repetidos para practica 1.2 y 1.3
12. Un reporte fue preparado

Resultados

Practica 1.1 - Programación de un semáforo

Arma el siguiente circuito y posteriormente realiza el código necesario para poner en marcha un semáforo. Sigue las instrucciones de la tabla para realizar el código.

Tabla 1.1 Secuencia del semáforo

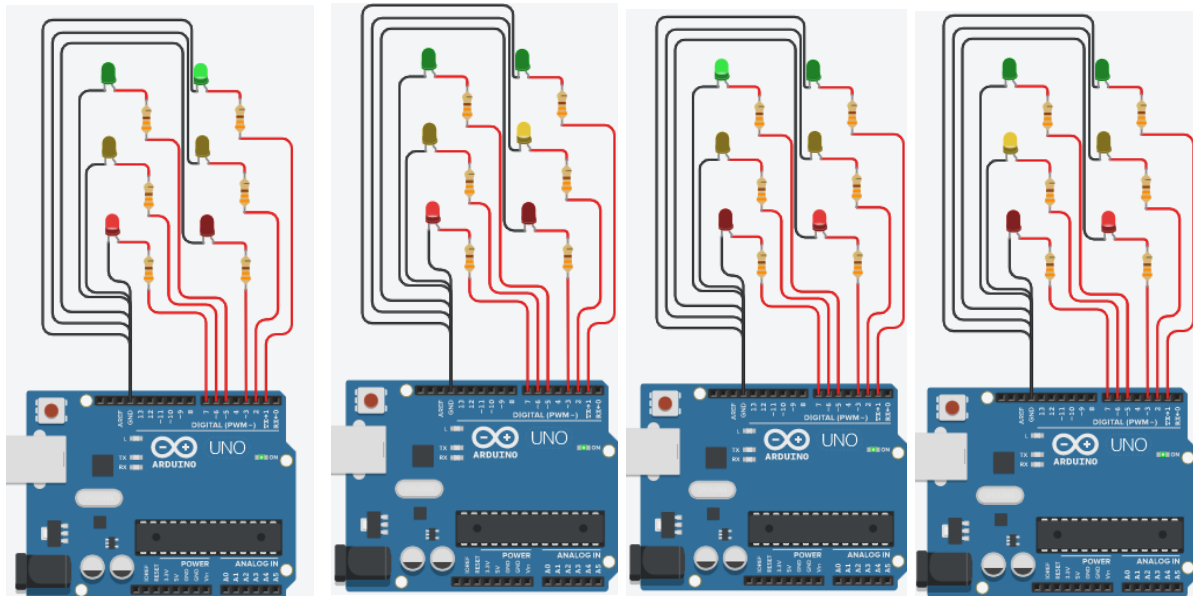
Semáforo 1 Semáforo 2 	Primer paso: • El LED rojo del semáforo 1 debe mantenerse encendido en este paso. • El LED verde del semáforo 2 debe de mantenerse encendido durante 5 segundos, luego parpadeará 5 veces con intermitencias de 500 ms para después apagarse.
Semáforo 1 Semáforo 2 	Segundo paso: • El LED rojo del semáforo 1 debe mantenerse encendido en este paso. • El LED amarillo del semáforo 2 debe mantenerse encendido durante 3 segundos y luego apagarse.
Semáforo 1 Semáforo 2 	Tercer paso: • Se enciende el LED rojo del semáforo 2 y se mantiene encendido. • Se apaga el LED rojo del semáforo 1 y se enciende el LED verde durante 5 segundos, luego parpadeará 5 veces con intermitencias de 500 ms para después apagarse, y así hasta repetir la secuencia que realizó el semáforo 2. • Repita el ciclo infinidad de veces o un mínimo de 10 transiciones.

Reporta el código realizado y el circuito implementado.

El código:

```
1 // C++ code
2 //
3 void setup()
4 {
5   pinMode(1, OUTPUT);
6   pinMode(7, OUTPUT);
7
8   pinMode(2, OUTPUT);
9   pinMode(3, OUTPUT);
10  pinMode(5, OUTPUT);
11  pinMode(6, OUTPUT);
12 }
13
14 void loop()
15 {
16   digitalWrite(1, HIGH);
17   digitalWrite(7, HIGH);
18   delay(5000); // Wait for 5000 millisecond(s)
19   digitalWrite(1, LOW);
20   delay(500); // Wait for 500 millisecond(s)
21   digitalWrite(1, HIGH);
22   delay(500); // Wait for 500 millisecond(s)
23   digitalWrite(1, LOW);
24   delay(500); // Wait for 500 millisecond(s)
25   digitalWrite(1, HIGH);
26   delay(500); // Wait for 500 millisecond(s)
27   digitalWrite(1, LOW);
28   delay(500); // Wait for 500 millisecond(s)
29   digitalWrite(1, HIGH);
30   delay(500); // Wait for 500 millisecond(s)
31   digitalWrite(1, LOW);
32   delay(500); // Wait for 500 millisecond(s)
33   digitalWrite(1, HIGH);
34   delay(500); // Wait for 500 millisecond(s)
35   digitalWrite(1, LOW);
36   delay(500); // Wait for 500 millisecond(s)
37   digitalWrite(1, HIGH);
38   delay(500); // Wait for 500 millisecond(s)
39   digitalWrite(1, LOW);
40
41   digitalWrite(2, HIGH);
42   delay(3000); // Wait for 3000 millisecond(s)
43   digitalWrite(2, LOW);
44   digitalWrite(3, HIGH);
45   digitalWrite(7, LOW);
46   digitalWrite(5, HIGH);
47   delay(5000); // Wait for 5000 millisecond(s)
48   digitalWrite(5, LOW);
49   delay(500); // Wait for 500 millisecond(s)
50   digitalWrite(5, HIGH);
51   delay(500); // Wait for 500 millisecond(s)
52   digitalWrite(5, LOW);
53   delay(500); // Wait for 500 millisecond(s)
54   digitalWrite(5, HIGH);
55   delay(500); // Wait for 500 millisecond(s)
56   digitalWrite(5, LOW);
57   delay(500); // Wait for 500 millisecond(s)
58   digitalWrite(5, HIGH);
59   delay(500); // Wait for 500 millisecond(s)
60   digitalWrite(5, LOW);
61   delay(500); // Wait for 500 millisecond(s)
62   digitalWrite(5, HIGH);
63   delay(500); // Wait for 500 millisecond(s)
64   digitalWrite(5, LOW);
65   delay(500); // Wait for 500 millisecond(s)
66   digitalWrite(5, HIGH);
67   delay(3000); // Wait for 3000 millisecond(s)
68   digitalWrite(6, HIGH);
69   delay(3000); // Wait for 3000 millisecond(s)
70   digitalWrite(6, LOW);
71 }
```

La simulación:



Practica 1.2 – Secuencia libre

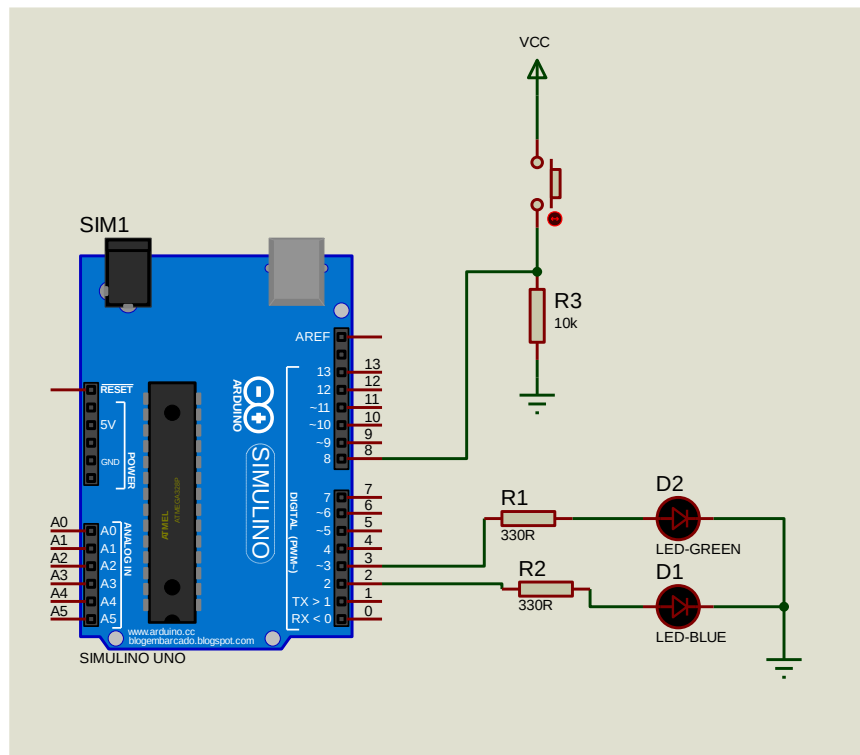
Con el circuito de la práctica 1.1, realiza un código que realice una secuencia libre con seis LEDs.

Reporta el código realizado y el circuito implementado.

Practica 1.3 – Lectura de un puerto

Implementa el siguiente circuito. Posteriormente al presionar un push-button debe activarse el parpadeo del LED azul, debe parpadear cinco veces con intermitencias de 500 ms. Al soltar el botón debe mantenerse encendido el LED verde de manera constante.

Reporta el código realizado y el circuito implementado.



Discusión

Conclusión

PRÁCTICA II- Comunicación Serial y LED RGB(Red-Blue-Green)



Objetivo de la unidad

Describir el funcionamiento de los comandos de programación, los microcontroladores, y los componentes asociados, así como el manejo de los mismos

Objetivos de la práctica

- Identificar los elementos del lenguaje de programación usados en la comunicación serial.
- Establecer la comunicación con el hardware abierto utilizando tecnología serial.

Comunicación Serial

Existen 2 formas de comunicación binaria, la **paralela y el serial**. La comunicación paralela se encarga de enviar los datos simultáneamente a través de 4 hilos, lo cual presenta su principal ventaja ya que la transferencia de datos es más rápida, pero el problema es que un cable por cada bit de dato, lo cual encarece y dificulta el diseño de placas, otro inconveniente es la capacitancia que generan los conductores por lo que la transmisión se vuelve deficiente a los pocos metros.

La comunicación serial es más lenta debido a que transmite bit por bit, pero tiene la ventaja de necesitar menos cantidad de hilos, y también puede extender la comunicación a mayor distancia, por ejemplo, en la norma RS232 a 15 m, en la norma RS422/485 a 1200 m y podemos utilizar modem para extenderlo a cualquier parte del mundo. La diferencia en cuanto a la cantidad de hilos requerido por ambas estrategias de comunicación es demostrada en la figura 6.

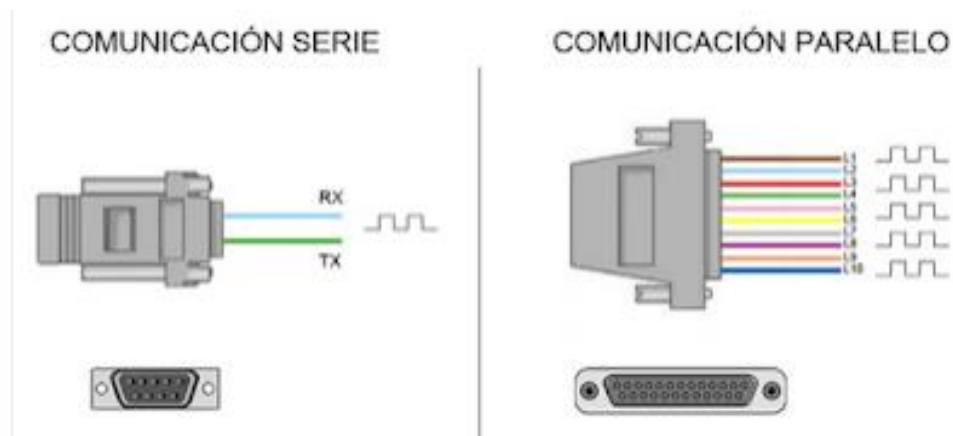


Figura 6.

de la diferencia en cantidad de hilos entre comunicación serie y comunicación paralelo

Ilustración

Existen 2 formas de realizar la comunicación serial: La sincrónica y la asíncrona, la diferencia que hay entre estas 2 formas de comunicación es que la sincrónica además de la línea de transmisión de datos necesita otra que contenga los pulsos de reloj, estos a su vez indican cuando un dato es válido.

El serial asíncrono no necesita pulsos de reloj, en su lugar utiliza mecanismo como referencia tierra (RS232) o voltajes diferenciales (RS433/485), donde la duración de cada bit es determinada por la velocidad de transmisión de datos que se debe definir previamente en cada equipo.

Comunicación Serial en Arduino

Los puertos de comunicación serial nos proporcionan la forma más efectiva de comunicar con los microcontroladores tipo Arduino con el ordenador, y como podrás notarlo a través de esta comunicación podremos mandar diferentes órdenes a nuestro Arduino para automatizar procesos o incluso recibir informaciones importantes para mostrarlas en la pantalla de nuestro computador.

La comunicación serial entre dos dispositivos únicamente utiliza 3 líneas las cuales son, véase también la ilustración 7:

- Línea de recepción de datos (RX)
- Línea de transmisión de datos (TX)
- Línea común (GND)

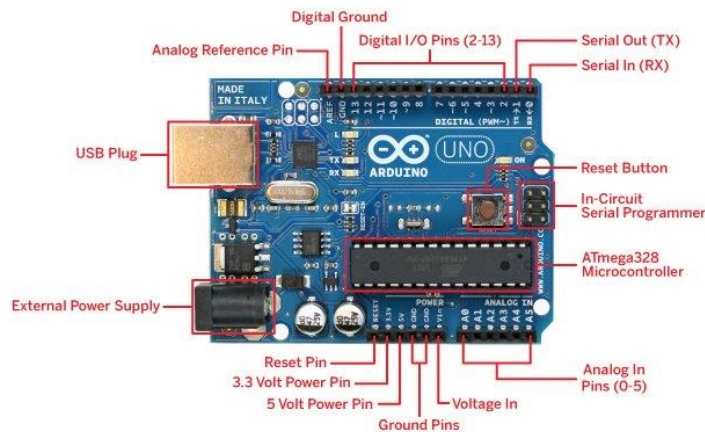


Figura 7. Diagrama pinout de un microcontrolador Arduino 1

El GND, el RX y TX del Arduino son fácilmente identificables en la placa.

Arduino UNO y Mini Pro: Pines 0 (RX) y 1 (TX);

Arduino Mega y Arduino Due: son cuatro puertos de serie.

puerto serie 0: pines 0 (RX) y 1 (TX)

puerto serie 1: pines 19 (RX) y 18 (TX)

puerto serie 2: pines 17 (RX) y 16 (TX)

puerto serie 3: pines 15 (RX) y 14 (TX)

El TX y RX del Arduino son los dos pines que emplea el dispositivo para realizar la comunicación por medio del protocolo serial. Los datos, por lo tanto, son transmitidos en la

línea o pin TX y son recibidos por la línea o pin RX. El esquema general de este cableado este ilustrado en la figura 8.

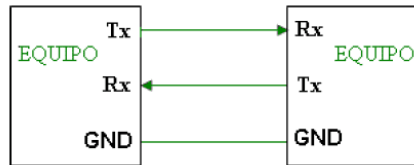


Figura 8. Ilustración del enlace entre pines de comunicación serial

Puerto serie del Arduino:

Las placas de Arduino poseen unidades UART que operan a nivel TTL 0 / 5v, lo que las vuelve compatibles con la conexión USB.

Como ya vimos los pines de los puertos seriales, podemos notar también que la mayoría de los microcontroladores Arduino disponen de un conector USB que nos permite una conexión serie instantánea con nuestro ordenador.

PUEDES CONSULTAR LOS DIFERENTES COMANDOS PARA LA COMUNICACIÓN SERIAL EN EL SIGUIENTE ENLACE:

<https://controlautomaticoeducacion.com/arduino/comunicacion-serial-con-arduino/>

Software Necesario

- TinkerCAD en línea (<https://www.tinkercad.com/>)
- Computadora con Windows o iOS

Materiales

- 1 Protoboard
- Cables de conexión.
- 1 Módulo RGB CNT1
- 1 Arduino Uno

Desarrollo de la práctica

1. La práctica 1 fue leída y comprendida
2. Un buscador fue utilizado para llegar a la página web de TinkercAD
3. El botón de circuitos fue pulsado. La ubicación del botón es demostrada en la figura 9

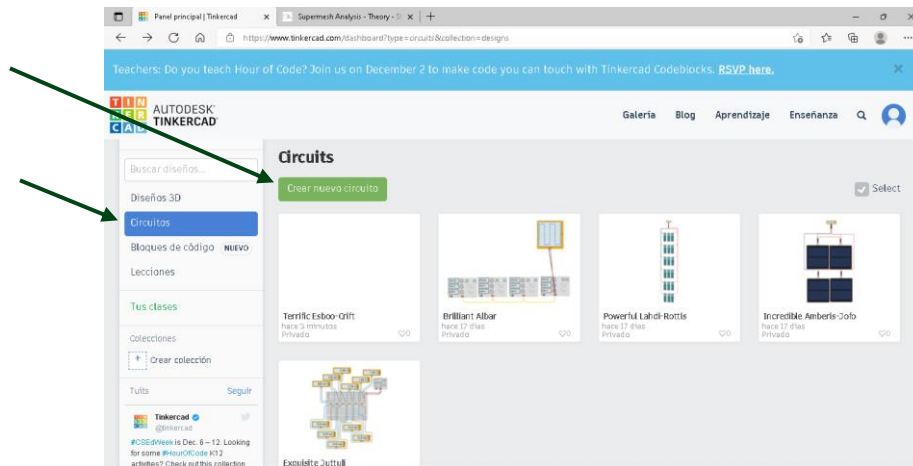


Figura 9. Seleccionar la opción de nuevo circuito

4. La opción de crear un nuevo circuito fue tomada pulsando el botón nombrado “Crear nuevo circuito” cuya ubicación es demostrado en la figura 9
5. La barra de componentes fue cambiada de componentes “básicos” a “todos”. La celda donde este cambio es hecho está indicada en la ilustración 10

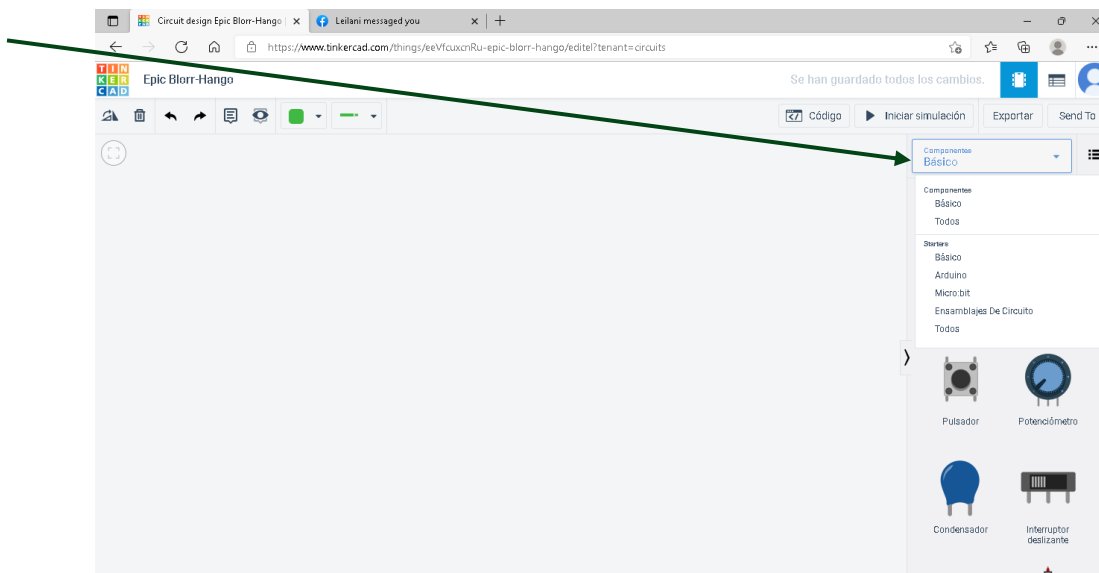


Figura 10. Selección disponibilidad de componentes

6. Los componentes requeridos para la práctica uno fueron seleccionados y puestos en el área de trabajo
7. El circuito exigido por el ejercicio 1 fue construido como es indicado en la figura 11

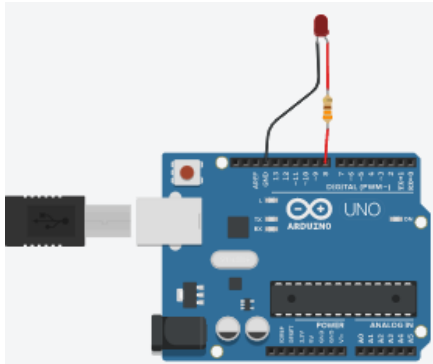


Figura 11. El circuito del ejercicio 1

8. La ventana para escribir código fue abierta picando el botón nombrado “Código”. La ubicación de este en la pantalla es tal como lo demuestra figura 12

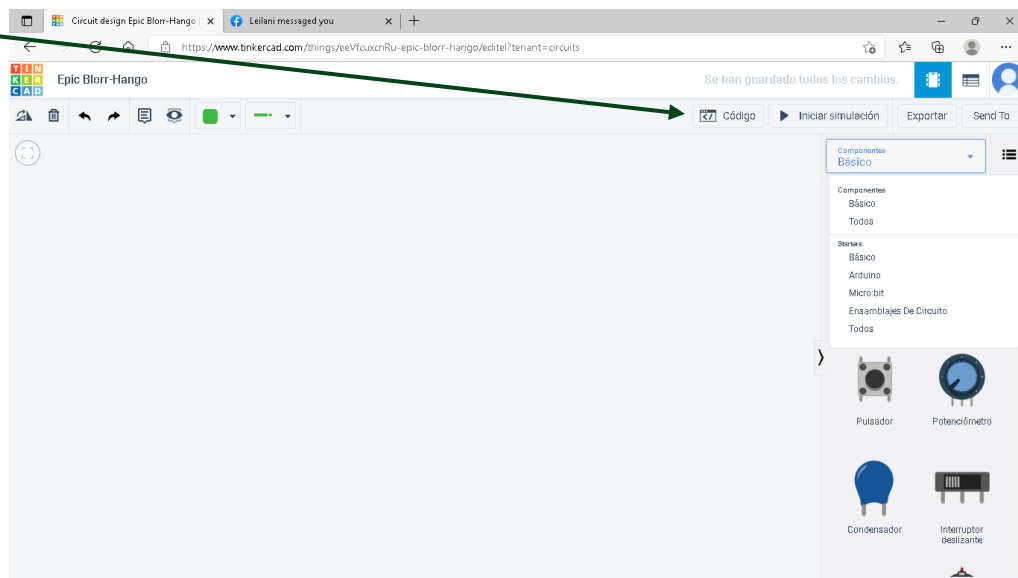


Figura 12. Pestaña para programar el Arduino

9. El código fue preparado según las instrucciones indicado en el ejercicio 1. La ventana de programación es demostrada en la figura 13

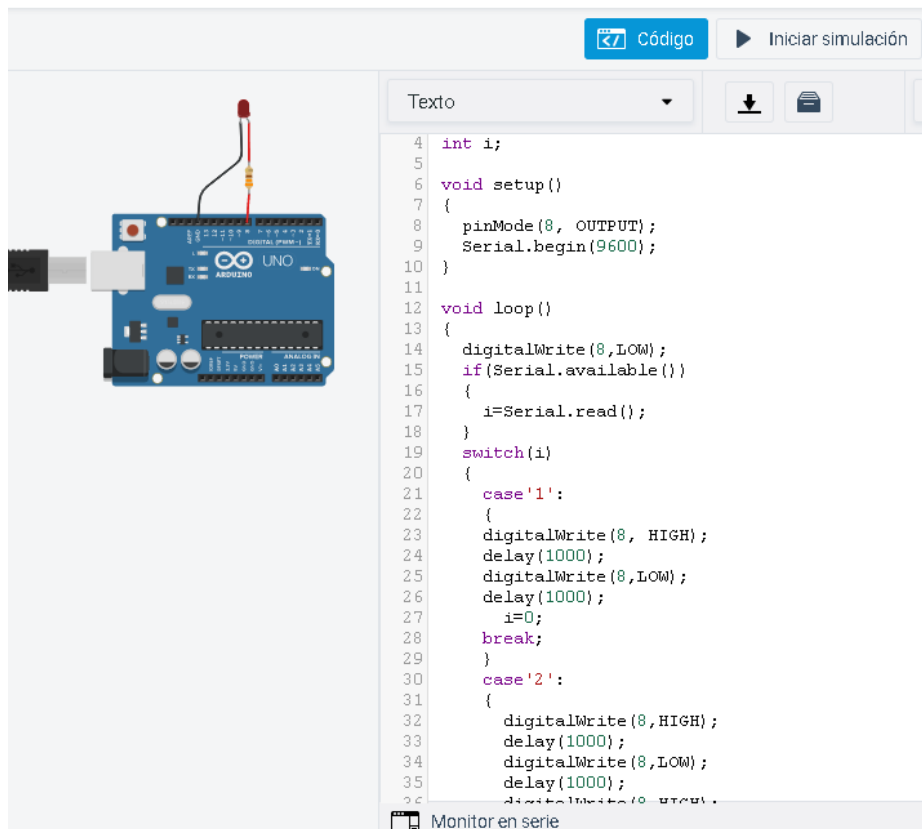


Figura 13. Ventana de programación

10. Los resultados de la práctica fueron documentados mediante capturas de pantalla en la sección de resultados
11. Pasos 1 a 10 fueron repetidos para práctica 2 y 3
12. Un reporte fue preparado

Resultados

Ejercicio 1. Parpadeo de un LED por comunicación serial.

Instrucciones: Realiza un programa el cual envíe números entre 1 a 9 a través del monitor serial, y la placa ARDUINO hará parpadear un LED el número de veces indicado. El parpadeo debe de tener intermitencias de 1 segundo y va conectado a cualquier puerto que tu elijas. Recuerda conectarle su resistencia de 330 Ohm.

El código:

```
1  int i;
2
3  void setup()
4  {
5      pinMode(8, OUTPUT);
6      Serial.begin(9600);
7  }
8
9  void loop()
10 {
11     digitalWrite(8,LOW);
12     if(Serial.available())
13     {
14         i=Serial.read();
15     }
16     switch(i)
17     {
18         case '1':
19         {
20             digitalWrite(8, HIGH);
21             delay(1000);
22             digitalWrite(8,LOW);
23             delay(1000);
24             i=0;
25             break;
26         }
27         case '2':
28         {
29             digitalWrite(8,HIGH);
30             delay(1000);
31             digitalWrite(8,LOW);
32             delay(1000);
33             i=0;
34             break;
35         }
36         case '3':
37         {
38             digitalWrite(8,HIGH);
39             delay(1000);
40             digitalWrite(8,LOW);
41             delay(1000);
42             digitalWrite(8,HIGH);
43             delay(1000);
44             digitalWrite(8, LOW);
45             delay(1000);
46             digitalWrite(8, HIGH);
47             delay(1000);
48             digitalWrite(8, LOW);
49             delay(1000);
50             digitalWrite(8,LOW);
51             i=0;
52             break;
53         }
54         case '4':
55         {
56             digitalWrite(8,HIGH);
57             delay(1000);
58             digitalWrite(8,LOW);
59             delay(1000);
60             digitalWrite(8,HIGH);
61             delay(1000);
62             digitalWrite(8,LOW);
63             delay(1000);
64             digitalWrite(8,HIGH);
65             delay(1000);
66             digitalWrite(8,LOW);
67             delay(1000);
68             digitalWrite(8,HIGH);
69             delay(1000);
70             digitalWrite(8,LOW);
71             delay(1000);
72             i=0;
73             break;
74         }
75         case '5':
76         {
77             digitalWrite(8,HIGH);
78             delay(1000);
79             digitalWrite(8,LOW);
80             delay(1000);
81             digitalWrite(8,HIGH);
82             delay(1000);
83             digitalWrite(8,LOW);
84             delay(1000);
85             digitalWrite(8,HIGH);
86             delay(1000);
87             digitalWrite(8,LOW);
88             delay(1000);
89             digitalWrite(8,HIGH);
90             delay(1000);
91             digitalWrite(8,LOW);
92             delay(1000);
93             digitalWrite(8,HIGH);
94             delay(1000);
95             digitalWrite(8,LOW);
96             delay(1000);
97             i=0;
98             break;
99         }
100        case '6':
101        {
102            digitalWrite(8,HIGH);
103            delay(1000);
104            digitalWrite(8,LOW);
105            delay(1000);
106            digitalWrite(8,HIGH);
107            delay(1000);
108            digitalWrite(8,LOW);
109            delay(1000);
110            digitalWrite(8,HIGH);
111            delay(1000);
112            digitalWrite(8,LOW);
113            delay(1000);
114            digitalWrite(8,HIGH);
115            delay(1000);
116            digitalWrite(8,LOW);
117            delay(1000);
118            digitalWrite(8,HIGH);
119            delay(1000);
120            digitalWrite(8,LOW);
121            delay(1000);
122            digitalWrite(8,HIGH);
123            delay(1000);
124            digitalWrite(8,LOW);
125            delay(1000);
126            i=0;
127            break;
128        }
129
130        case '7':
131        {
132            digitalWrite(8,HIGH);
133            delay(1000);
134            digitalWrite(8,LOW);
135            delay(1000);
136            digitalWrite(8,HIGH);
137            delay(1000);
138            digitalWrite(8,LOW);
139            delay(1000);
140            digitalWrite(8,HIGH);
141            delay(1000);
142            digitalWrite(8,LOW);
143            delay(1000);
144            digitalWrite(8,HIGH);
145            delay(1000);
146            digitalWrite(8,LOW);
147            delay(1000);
148            digitalWrite(8,HIGH);
149            delay(1000);
150            digitalWrite(8,HIGH);
151            delay(1000);
152            digitalWrite(8,LOW);
153            delay(1000);
154            digitalWrite(8,HIGH);
155            delay(1000);
156            digitalWrite(8,LOW);
157            delay(1000);
158            i=0;
159            break;
160
161        }
162        case '8':
163        {
164            digitalWrite(8,HIGH);
165            delay(1000);
166            digitalWrite(8,LOW);
167            delay(1000);
168            digitalWrite(8,HIGH);
169            delay(1000);
170            digitalWrite(8,LOW);
171            delay(1000);
172            digitalWrite(8,HIGH);
173            delay(1000);
174            digitalWrite(8,LOW);
175            delay(1000);
176            digitalWrite(8,HIGH);
177            delay(1000);
178            digitalWrite(8,LOW);
179            delay(1000);
180            digitalWrite(8,HIGH);
181            delay(1000);
182            digitalWrite(8,LOW);
183            delay(1000);
184            digitalWrite(8,HIGH);
185            delay(1000);
186            digitalWrite(8,LOW);
187            delay(1000);
188            digitalWrite(8,HIGH);
189            delay(1000);
190            digitalWrite(8,LOW);
191            delay(1000);
192            digitalWrite(8,HIGH);
```

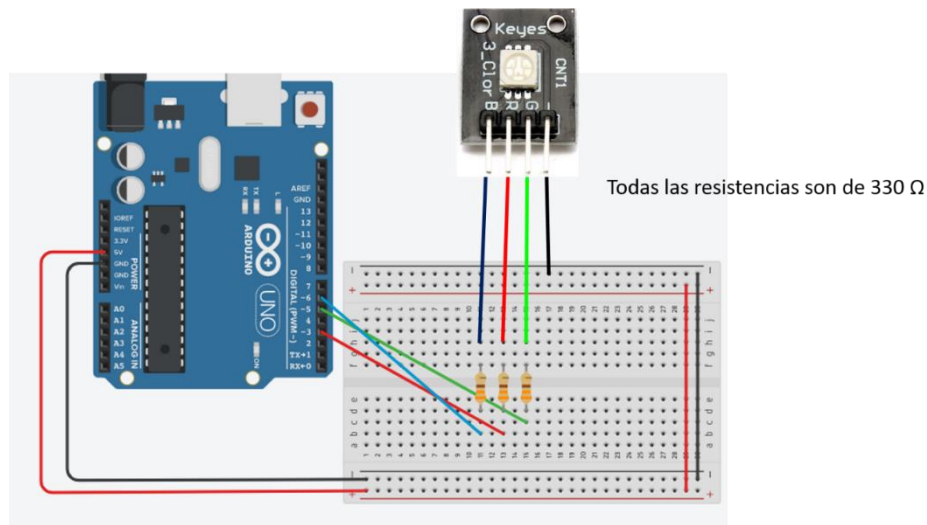
```

193     delay(1000);
194     digitalWrite(8,LOW);
195     delay(1000);
196     i=0;
197     break;
198 }
199 case '9':
200 {
201     digitalWrite(8,HIGH);
202     delay(1000);
203     digitalWrite(8,LOW);
204     delay(1000);
205     digitalWrite(8,HIGH);
206     delay(1000);
207     digitalWrite(8,LOW);
208     delay(1000);
209     digitalWrite(8,HIGH);
210     delay(1000);
211     digitalWrite(8,LOW);
212     delay(1000);
213     digitalWrite(8,HIGH);
214     delay(1000);
215     digitalWrite(8,LOW);
216     delay(1000);
217     digitalWrite(8,HIGH);
218     delay(1000);
219     digitalWrite(8,LOW);
220     delay(1000);
221     digitalWrite(8,HIGH);
222     delay(1000);
223     digitalWrite(8,LOW);
224     delay(1000);
225     digitalWrite(8,HIGH);
226     delay(1000);
227     digitalWrite(8,LOW);
228     delay(1000);
229     digitalWrite(8,HIGH);
230     delay(1000);
231     digitalWrite(8,LOW);
232     delay(1000);
233     digitalWrite(8,HIGH);
234     delay(1000);
235     digitalWrite(8,LOW);
236     delay(1000);
237     i=0;
238     break;
239 }
240 }
241
242
243 }

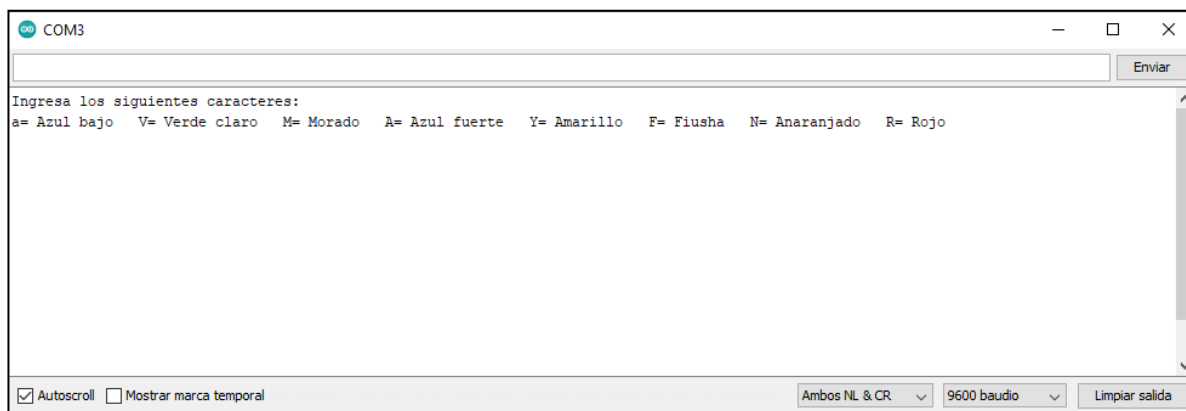
```

Ejercicio 2. Control de LED RGB por comunicación Serial

Instrucciones: Implementa el siguiente circuito.



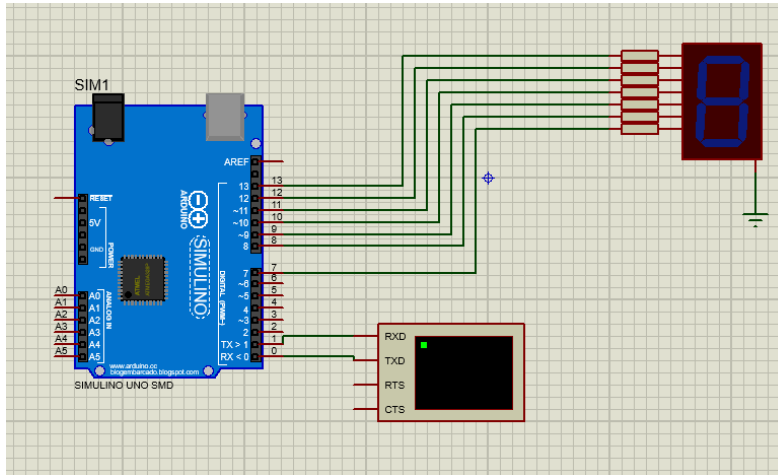
Posteriormente realiza un programa que muestre el siguiente mensaje. Al ingresar el carácter correspondiente en el monitor serial, el LED RGB debe mostrar el color que corresponde.



Ejercicio 3. Control de Display de 7 segmentos con comunicación Serial

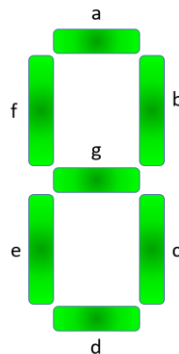
Instrucciones: Implementa el siguiente circuito en Proteus y realiza un programa que al enviar los numero de 0-9 a través del monitor serial se puedan visualizar en el display de 7 segmentos de cátodo común.

Las resistencias del diagrama son de 330 Ohm.



LISTA DE COMPONENTES EN PROTEUS Y PALABRAS CLAVE			
CANTIDAD	NOMBRE DEL COMPONENTE	DESCRIPCIÓN	PALABRA CLAVE PARA BUSQUEDA
1	7SEG-COM-CAT-BLUE	Blue, 7-segment Common Anode	DISPLAY

Recuerda como es la estructura de un display de 7 segmentos.



Discusión

Conclusión

PRÁCTICA III- Lectura de un puerto de entrada



Objetivo de la unidad:

Describir el funcionamiento de los comandos de programación, los microcontroladores, y los componentes asociados, así como el manejo de los mismos

Objetivos de la práctica:

- El alumno practicará la programación de microcontroladores
- El alumno practicará conexión de microcontroladores con entrada por puerto
- El alumno comprenderá la importancia de entrada por puerto a los microcontroladores

Antecedentes

Los microcontroladores tal como los de Arduino son aptos para la elaboración de operaciones matemáticas con la finalidad de producir una respuesta mediante cualquier dispositivo periférico perteneciente al grupo nombrado actuadores. Varios esquemas existen para el control de actuadores en los cuales el microcontrolador recibe algún tipo de información sobre que tipo y que magnitud de acción debe ser tomada. Para esto, los microcontroladores están equipados con puertos que pueden funcionar como entrada de datos o información al sistema.

Un esquema sumamente común para ingresar información a un microcontrolador es el denominado push-button. Los push-buttons pueden ser conectados en uno de dos arreglos que viene siendo el arreglo, pull-down o pull-up. En ambos esquemas el concepto es de provocar un cambio de voltaje que llega al puerto de entrada al cual esta conectado. El cambio de voltaje es detectado por el puerto y interpretado como un cambio de posición on/off. Instrucciones para la interpretación de estos cambios de posición son embebidos, así como las acciones a tomar en respuesta a las entradas.

Figura 14 ilustra esquemáticamente la diferencia entre los arreglos pull-up/ pull-down.

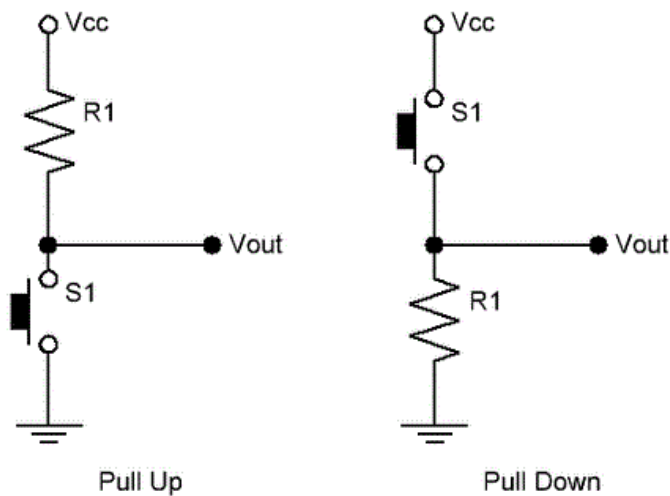


Figura 14. Ilustración esquemática de arreglos pull-up y pull-down

Software necesario

- TinkerCAD en línea (<https://www.tinkercad.com/>)
- Computadora con Windows o iOS

Desarrollo de la práctica

1. La práctica 1 fue leída y comprendida
2. Un buscador fue utilizado para llegar a la página web de TinkerCAD

3. El botón de circuitos fue pulsado. La ubicación del botón es demostrada en la figura 15

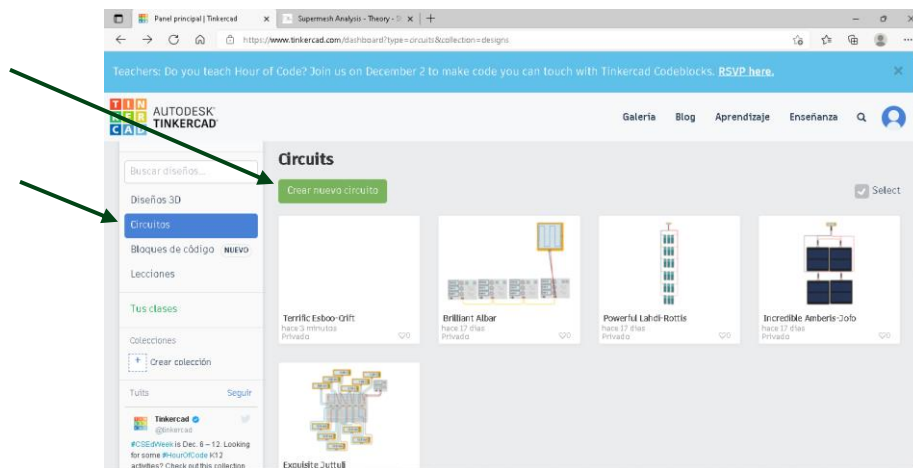


Figura 15. Seleccionar la opción de nuevo circuito

4. La opción de crear un nuevo circuito fue tomada pulsando el botón nombrado “Crear nuevo circuito” cuya ubicación es demostrado en la figura 15
5. La barra de componentes fue cambiada de componentes “básicos” a “todos”. La celda donde este cambio es hecho está indicada en la ilustración 16

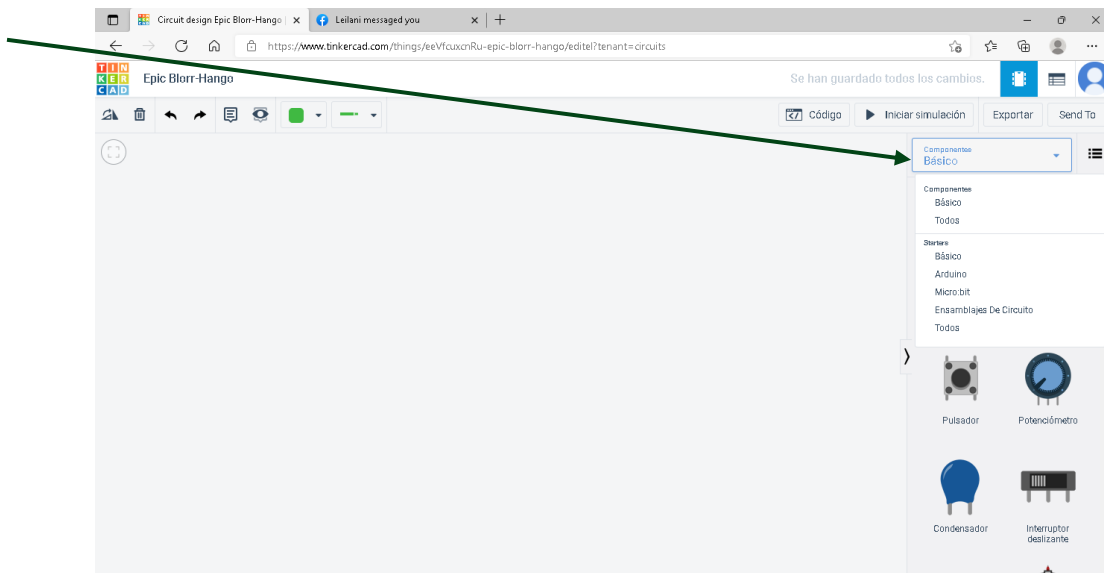


Figura 16. Selección disponibilidad de componentes

6. Los componentes requeridos para la práctica uno fueron seleccionados y puestos en el área de trabajo
7. El circuito exigido por el ejercicio fue construido como es indicado en la figura 17

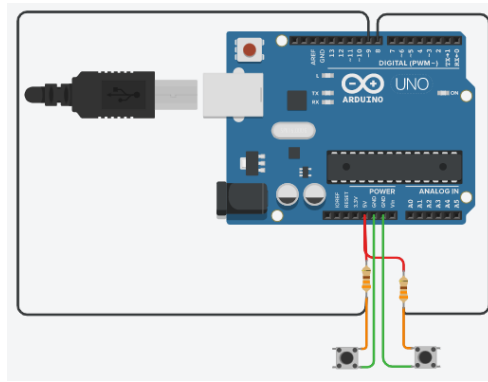


Figura 17. Circuito con dos push-buttons en arreglo pull-up

8. La ventana para escribir código fue abierta picando el botón nombrado “Código”. La ubicación de este en la pantalla es tal como lo demuestra figura 18

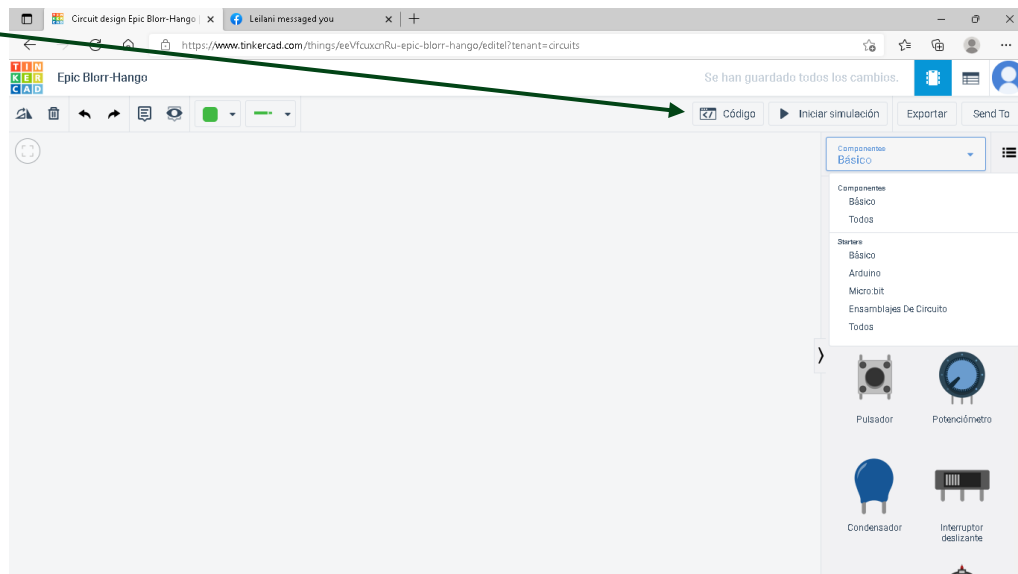


Figura 18. Pestaña para programar el Arduino

9. El código fue preparado según las instrucciones indicado en el ejercicio. La ventana de programación es demostrada en la figura 19

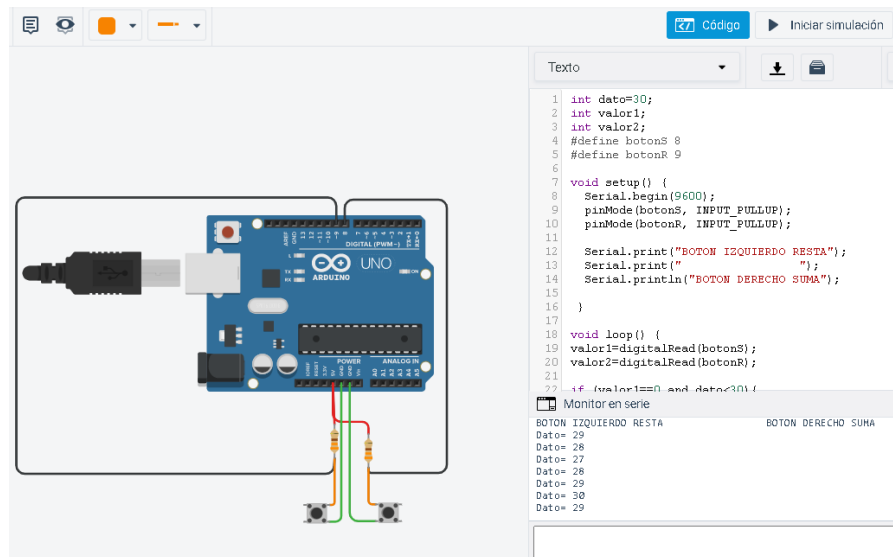


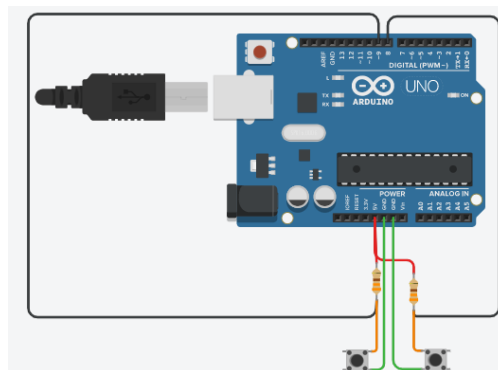
Figura 19. Ventana de programación

10. Los resultados de la practica fueron documentados mediante capturas de pantalla en la sección de resultados
11. Un reporte fue preparado

Resultados

Construir el circuito con dos botones pull-up en cualquier puerto entrada del microcontrolador y escribir un programa que suma y resta cada vez que uno o el otro push-button es apretado. El contador debe iniciar en 30 y no puede contar menos de cero o mas de 30.

El Circuito:



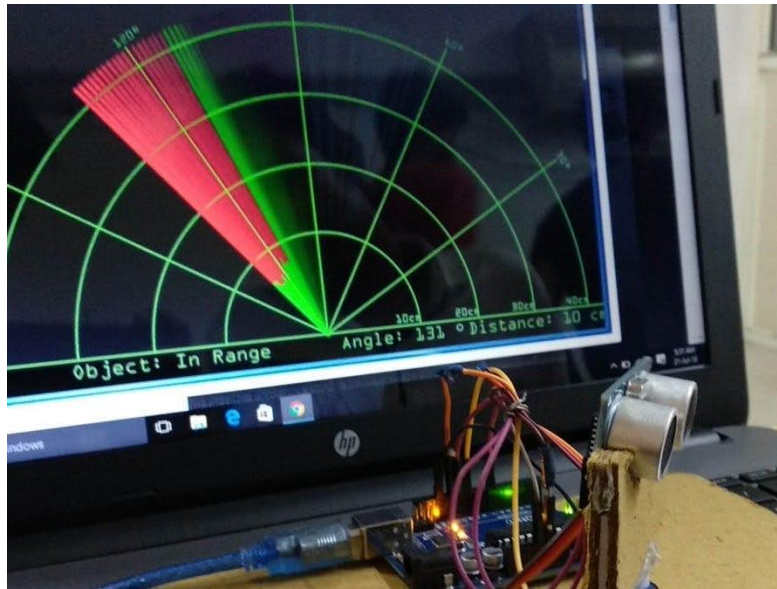
La Programa:

```
1  int dato=30;
2  int valor1;
3  int valor2;
4  #define botonS 8
5  #define botonR 9
6
7  void setup() {
8      Serial.begin(9600);
9      pinMode(botonS, INPUT_PULLUP);
10     pinMode(botonR, INPUT_PULLUP);
11
12     Serial.print("BOTON IZQUIERDO RESTA");
13     Serial.print(" ");
14     Serial.println("BOTON DERECHO SUMA");
15
16 }
17
18 void loop() {
19     valor1=digitalRead(botonS);
20     valor2=digitalRead(botonR);
21
22     if (valor1==0 and dato<30) {
23         dato=dato+1;
24         Serial.print("Dato= ");
25         Serial.println(dato);
26         delay(350);
27     }
28
29     if (valor2==0 and dato>0) {
30         dato=dato-1;
31         Serial.print("Dato= ");
32         Serial.println(dato);
33
34         delay(350);
35     }
36
37
38 }
```

Discusión

Conclusiones

PRÁCTICA IV- Sensor Ultrasónico (HC-SR04)



Objetivo de la unidad

Describir el funcionamiento de los comandos de programación, los microcontroladores, y los componentes asociados, así como el manejo de los mismos

Objetivos de la práctica

- Describir el uso de sensores digitales: movimiento y proximidad.

Antecedentes

Sensor ultrasónico HC-SR04

El sensor HC-SR04 es un sensor de distancia de bajo costo que utiliza ultrasonido para determinar la distancia de un objeto. Destaca por su pequeño tamaño, bajo consumo energético, buena precisión y excelente precio.

El sensor HC-SR04 es el más utilizado dentro de los sensores de tipo ultrasonido, principalmente por la cantidad de información y proyectos disponibles en la web. De igual forma es el más empleado en proyectos de robótica como robots laberinto o sumo, y en proyectos de automatización como sistemas de medición de nivel o distancia. Un ejemplar de el sensor HC-SR04 es demostrado en la figura 20



Figura 20. Sensor Ultrasónico HC-SR04

Funcionamiento general

El sensor ultrasónico HC-R04 envía una señal a una frecuencia de 40 kHz, el oído humano normal escucha en un rango de 20 Hz a 20 kHz. La señal ultrasónica viaja a una velocidad de 344 m/s en el aire. El sensor mide el tiempo en que se envió la señal y esta tarda en regresar (el eco). Similar al método que usan los murciélagos para saber dónde se encuentran los objetos, ya que ellos son prácticamente ciegos. El principio de funcionamiento es ilustrado en figura 21

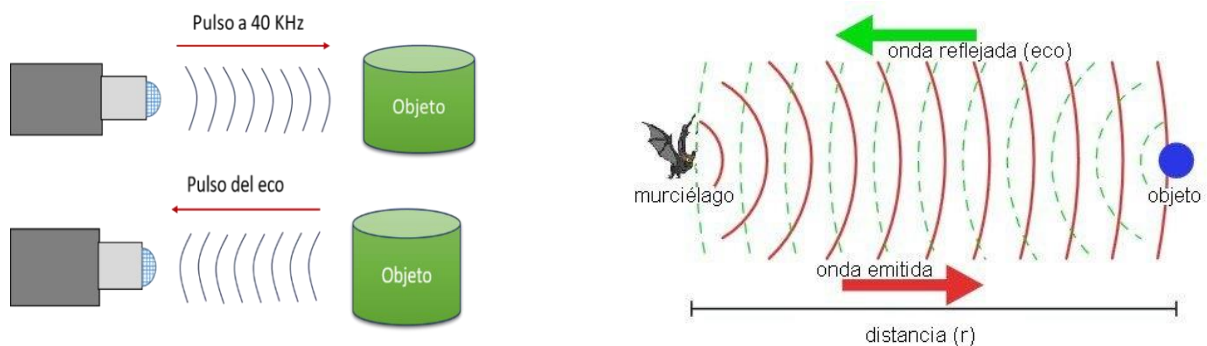


Figura 21. Principio de funcionamiento de sensor ultrasónico

Funcionamiento técnico

- El sensor manda una señal de arranque en el pin trigger (disparo) del SRF04 y después lee el ancho del impulso que nos proporciona el pin de eco (eco). El pulso de disparo tiene que tener un ancho mínimo de 10 μ s (por eso la velocidad se expresa en microsegundos).
- Después leemos el pulso de salida de eco y medimos su longitud que es proporcional al eco recibido. En caso de que no se produzca ningún eco, porque no se encuentra un objeto, el pulso de eco tiene una longitud aproximada de 36 ms.
- Hay que dejar un retardo de 10 ms desde que se hace una lectura hasta que se realiza la siguiente, con el fin de que el circuito se estabilice.

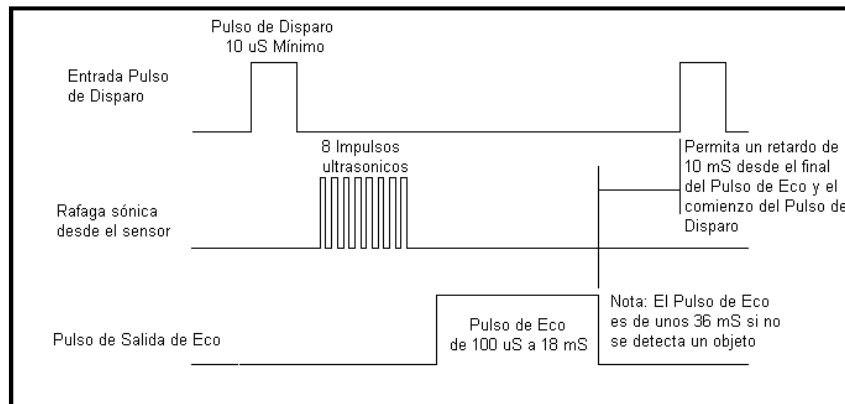


Figura 22. Funcionamiento teórico del sensor ultrasónico

Para la programación el fabricante recomienda las fórmulas en la figura 23 para hacer la conversión del ancho del pulso obtenido del sensor a unidades de distancia sean (cm o pies).

$$d = \frac{\text{duración del pulso}}{58} \quad \longrightarrow \quad \text{Fórmula para obtener los valores en cm}$$

$$d = \frac{\text{duración del pulso}}{148} \quad \longrightarrow \quad \text{Fórmula para obtener los valores en Inch (pulgadas)}$$

$$d = \frac{0.034 \cdot \text{duración del pulso}}{2} \quad \longrightarrow \quad \text{Fórmula para obtener los valores en cm}$$

Figura 23. Ecuaciones para la programación permitiendo el uso del HC-SR04

Terminales

- **Vcc:** Voltaje de alimentación de 5 V
- **Trig:** Terminal de disparo (se envía un pulso de 10 μ s para que empiece a enviar el ultrasonido. (Se configura en Arduino como OUTPUT).
- **Echo:** Terminal que regresa un pulso proporcional al tiempo medido. (Se configura en Arduino como OUTPUT).
- **Gnd:** Groud o tierra del sensor.

Características técnicas

Características técnicas de HC-SR04	
Rango	2 cm a 4 m
Alimentación	5 V a 30 mA
Frecuencia de señal emitida	40 KHz
Pulso de disparo	10 μ s min. TTL
Puso del Eco	100 μ s – 18 ms
Retardo entre pulsos	10 ms mínimo.
Angulo de medición	A 15° más eficaz, pero puede medir hasta 30°
Peso	10 gr
Tamaño	43 mm x 20 mm x 17 mm

Software Necesario

- TinkerCAD en línea (<https://www.tinkercad.com/>)
- Computadora con Windows o iOS

Desarrollo de la práctica

1. La práctica fue leída y comprendida
2. Un buscador fue utilizado para llegar a la página web de TinkerCAD
3. El botón de circuitos fue pulsado. La ubicación del botón es demostrada en la figura 24

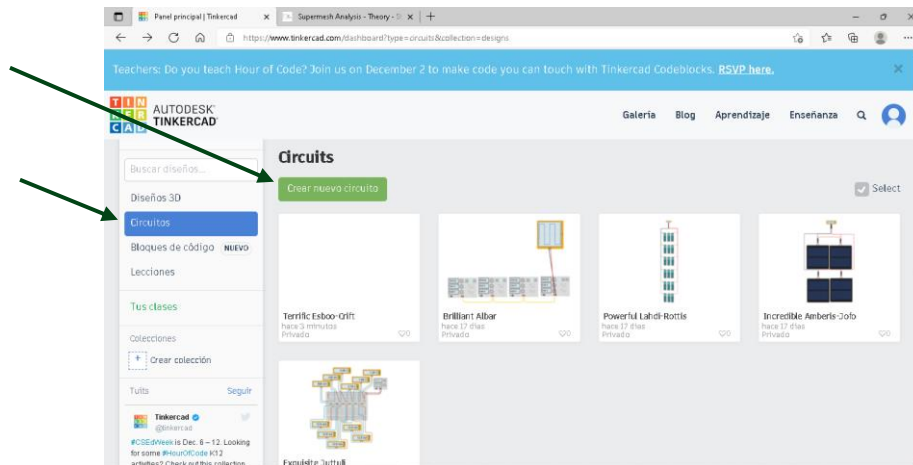


Figura 24. Seleccionar la opción de nuevo circuito

4. La opción de crear un nuevo circuito fue tomada pulsando el botón nombrado “Crear nuevo circuito” cuya ubicación es demostrado en la figura 24
5. La barra de componentes fue cambiada de componentes “básicos” a “todos”. La celda donde este cambio es hecho está indicada en la ilustración 25

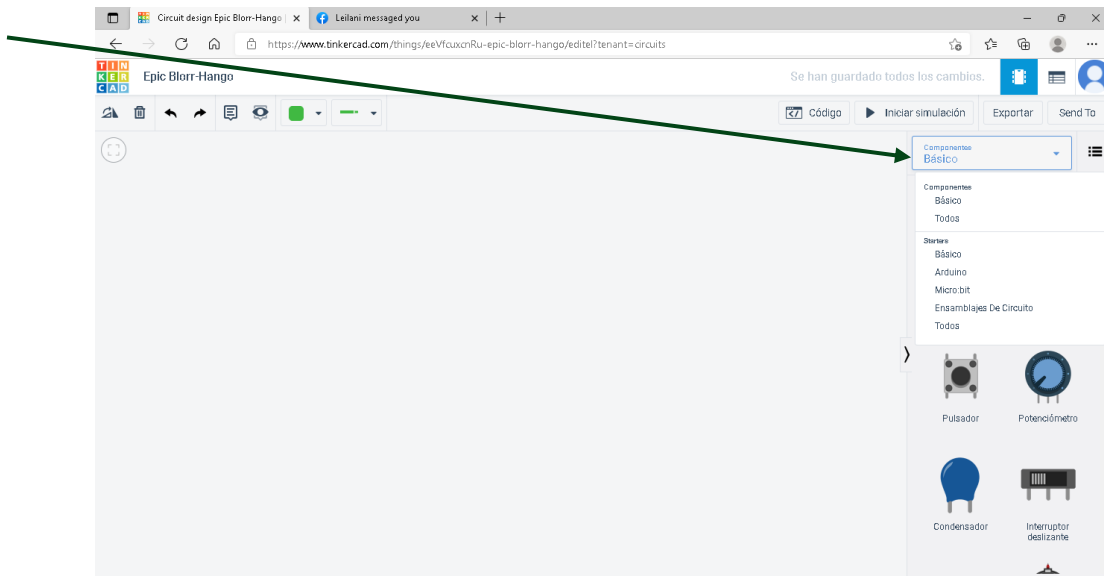


Figura 25. Selección disponibilidad de componentes

6. Los componentes requeridos para la práctica uno fueron seleccionados y puestos en el área de trabajo
7. El circuito exigido por el ejercicio fue construido como es indicado en la figura 26

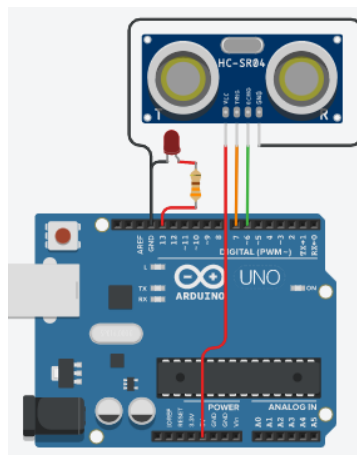


Figura 26. Circuito con sensor HC-SR04 y un led

8. La ventana para escribir código fue abierta picando el botón nombrado “Código”. La ubicación de este en la pantalla es tal como lo demuestra figura 27

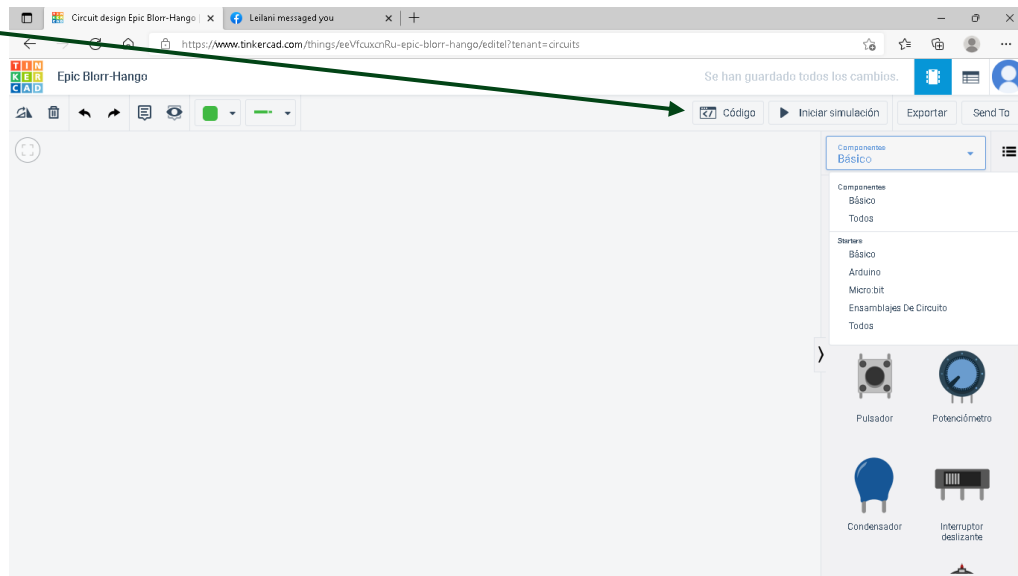


Figura 27. Pestaña para programar el Arduino

9. El código fue preparado según las instrucciones indicado en el ejercicio. La ventana de programación es demostrada en la figura 28

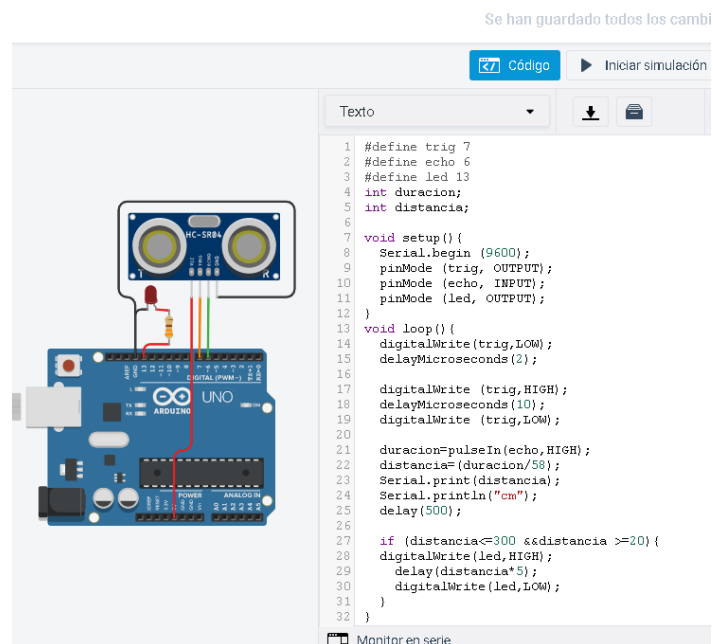


Figura 28. Ventana de programación

10. La simulación fue iniciada. Para simular el cambio de distancia, el cursor fue movido sobre la bocina del sensor y apretado, como lo ilustra la figura 29.

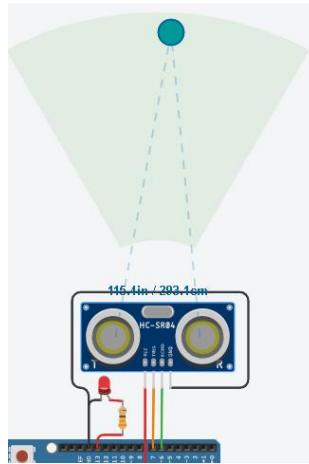


Figura 29. Simulación del cambio de distancia

11. El círculo oscuro fue movido dentro del cono indicado en la figura 29 simulando diferencias de distancias.
12. Los resultados de la practica fueron documentados mediante capturas de pantalla en la sección de resultados
13. Un reporte fue preparado

Resultados

```

1  #define trig 7
2  #define echo 6
3  #define led 13
4  int duracion;
5  int distancia;
6
7  void setup() {
8      Serial.begin (9600);
9      pinMode (trig, OUTPUT);
10     pinMode (echo, INPUT);
11     pinMode (led, OUTPUT);
12 }
13 void loop() {
14     digitalWrite(trig,LOW);
15     delayMicroseconds(2);
16
17     digitalWrite (trig,HIGH);
18     delayMicroseconds(10);
19     digitalWrite (trig,LOW);
20
21     duracion=pulseIn(echo,HIGH);
22     distancia=(duracion/58);
23     Serial.print(distancia);
24     Serial.println("cm");
25     delay(500);
26
27     if (distancia<=300 &&distancia >=20) {
28         digitalWrite(led,HIGH);
29         delay(distancia*5);
30         digitalWrite(led,LOW);
31     }
32 }

```

Monitor en serie

Discusión

Conclusión

PRÁCTICA V- Sensor PIR



Objetivo de la unidad

Describir el funcionamiento de los comandos de programación, los microcontroladores, y los componentes asociados, así como el manejo de los mismos

Objetivos de la práctica

- Al término de la actividad, el alumno podrá reforzar su conocimiento de algunos conceptos como: swap, memoria virtual, paginación, entre otros, en diferentes sistemas

Antecedentes

Software Necesario

- TinkerCAD en línea (<https://www.tinkercad.com/>)
- Computadora con Windows o iOS

Desarrollo de la práctica

1. La práctica 4.11 fue leída y comprendida
2. Un buscador fue utilizado para llegar a la página web de TinkerCAD
3. El botón de circuitos fue pulsado. La ubicación del botón es demostrada en la figura 21

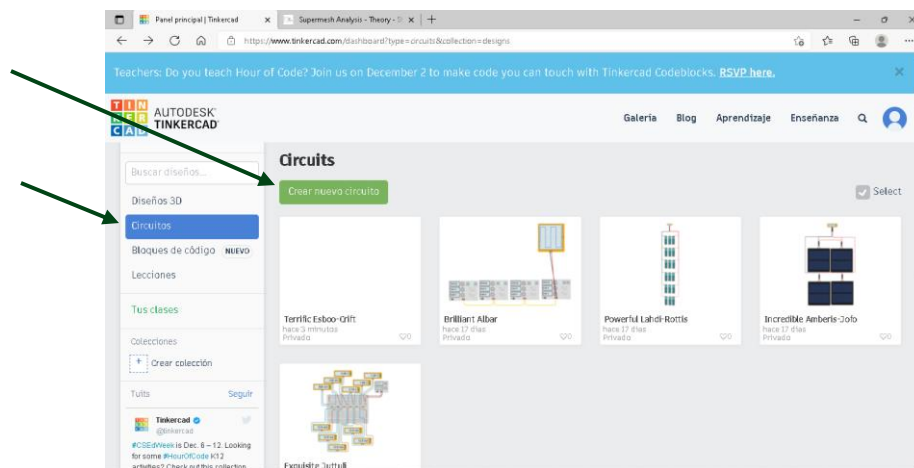


Figura 21. Seleccionar la opción de nuevo circuito

4. La opción de crear un nuevo circuito fue tomada pulsando el botón nombrado "Crear nuevo circuito" cuya ubicación es demostrado en la figura 21
5. La barra de componentes fue cambiada de componentes "básicos" a "todos". La celda donde este cambio es hecho está indicada en la ilustración 22

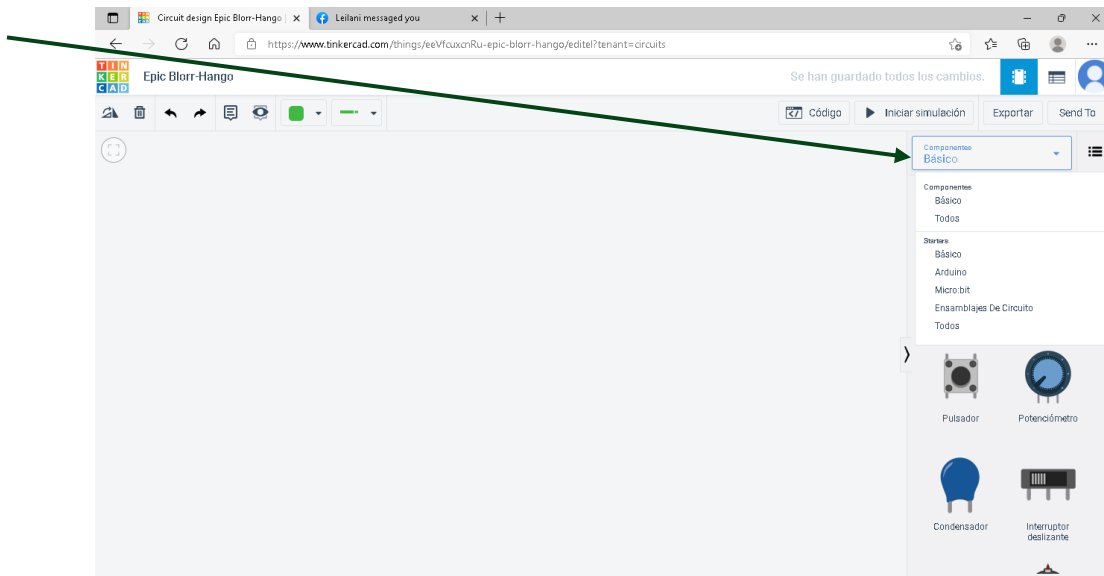


Figura 22. Selección disponibilidad de componentes

6. Los componentes requeridos para la práctica uno fueron seleccionados y puestos en el área de trabajo
7. El circuito exigido por el ejercicio 4.1 fue construido como es indicado en la figura 23

Figura 23. Circuito con tres led de diferentes colores controlado por un Arduino UNO

8. La ventana para escribir código fue abierta picando el botón nombrado “Código”. La ubicación de este en la pantalla es tal como lo demuestra figura 24

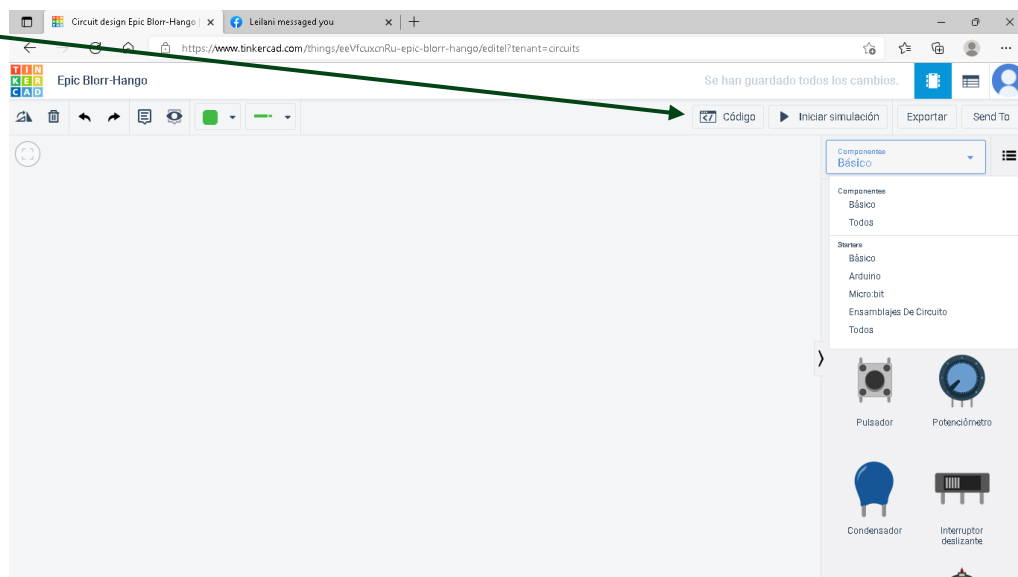


Figura 24. Pestaña para programar el Arduino

9. El código fue preparado según las instrucciones indicado en el ejercicio 4.1. La ventana de programación es demostrada en la figura 25

Figura 25. Ventana de programación

10. Los resultados de la practica fueron documentados mediante capturas de pantalla en la sección de resultados
11. Pasos 1-10 fueron repetidos para ejercicios 4.2 y 4.3
12. Un reporte fue preparado
- 13.

Resultados

Discusión

Conclusión

Bibliografía

- [1] Autores de Enciclopedias Britannica, «www.britannica.com,» Enciclopedias Britannica, 2 noviembre 2021. [En línea]. Available: <https://www.britannica.com/technology/ohmmeter>.

- [2] Fluke, «www.fluke.com,» Fluke (registered trade mark), 9 mayo 2021. [En línea]. Available: <https://www.fluke.com/en-us/learn/blog/electrical/what-is-resistance>. [Último acceso: 2 noviembre 2021].

- [3] W. Naeem, Concepts in Electric Circuits, Bookboon, 2009.

- [4] Autores de Enciclopedia Britannica, «www.britannica.com,» Eciclopedia Britannica, 22 agosto 2018. [En línea]. Available: <https://www.britannica.com/science/resistivity>. [Último acceso: 2 noviembre 2021].

- [5] Autores, Dictionario Merriam-Webster, «www.merriam-webster.com,» 2 noviembre 2021. [En línea]. Available: <https://www.merriam-webster.com/dictionary/voltage>.

- [6] Autores de Enciclopedia Britannica, «www.britannica.com,» 27 mayo 2020. [En línea]. Available: <https://www.britannica.com/science/electric-current>. [Último acceso: 2 noviembre 2021].

- [7] F. F. Mazda, Telecommunications Engineer's Reference Book, Oxford: Butterworth-Heinemann LTD, 1993.

- [8] Missouri University of Science and Technology, «web.mst.edu,» Periodic Signals, [En línea]. Available: <https://web.mst.edu/~kosbar/ee3430/ff/fourier/periodic.html>. [Último acceso: 7 noviembre 2021].

- [9] Editores de Dictionario Collings, «www.collinsdictionary.com,» peak-to-peak value, [En línea]. Available: <https://www.collinsdictionary.com/dictionary/english/peak-to-peak-value>. [Último acceso: 7 noviembre 2021].

- [10] F. Obeidat, «www.philadelphia.edu.jo,» EElectric Circuits II-Sinusoids and Phasors, [En línea]. Available: <https://www.philadelphia.edu.jo/academics/fobeidat/uploads/Electric%20Circuits%20I%20Course/2%20Sinusoids%20and%20phasor.pdf>. [Último acceso: 7 noviembre 2021].